MBLP131 · Hafta 3 · Ders Notları

İşletim Sistemleri Temelleri

İşletim Sistemi Hizmetleri, Sistem Çağrıları ve Süreçler

İşletim sistemi bir program için ne yapar ve çalışan bir program tam olarak nedir?

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. İşletim sisteminin kullanıcılara ve programlara sunduğu hizmetleri ve kendi verimli çalışması için ihtiyaç duyduğu hizmetleri saymak.
2. Bir API fonksiyonunu, bir kütüphane sarmalayıcısını ve bir sistem çağrısını birbirinden ayırmak; bir C programındaki tek bir `write()` çağrısını çekirdeğe kadar ve geri izlemek.
3. Sistem çağrısı tablosunu, parametre aktarmanın üç yolunu ve sistem çağrılarının altı kategorisini açıklamak.
4. Program ile süreç arasındaki farkı açıklamak ve bir sürecin bellek düzenini (text, data, bss, heap, stack) çizmek.
5. Beş durumlu süreç diyagramını çizmek, her geçişi neyin tetiklediğini söylemek ve bir süreç denetim bloğunun (process control block, PCB) içeriğini saymak.
6. Bağlam değişimini anlatıp ek yükünü hesaplamak; `fork()`, `exec()` ve `wait()` çağrılarını, zombileri ve yetimleri açıklamak; bir döngü içindeki `fork()` çağrılarının kaç süreç yarattığını saymak.

İçindekiler

1 İşletim Sistemi Hizmetleri 2

2 Ayrıntılı Sistem Çağrıları 3

3 Süreç Kavramı 9

4 Süreç Durumları 11

5 Süreç Denetim Bloğu 13

6 Bağlam Değişimi 14

7 Süreç Oluşturma ve Sonlandırma 15

8 Özet 19

9 Sözlük 20

10 Kendiniz Deneyin 21

11 Tekrar Soruları 21

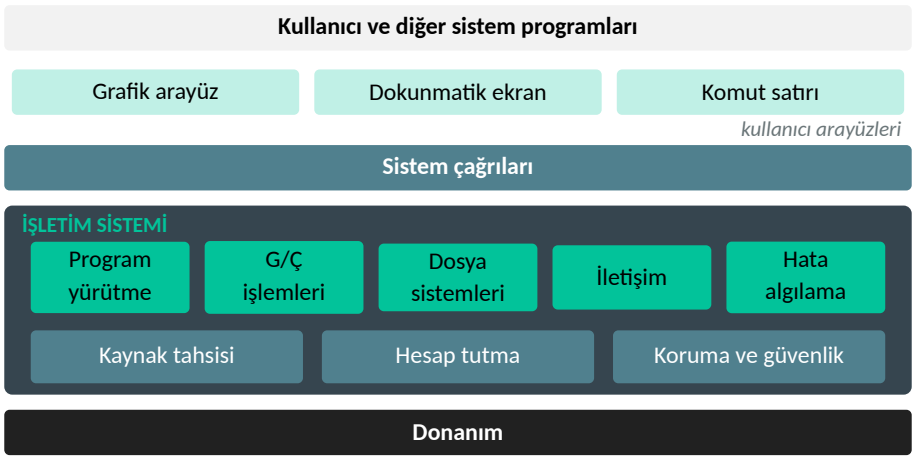
12 İleriye Bakış: 4. Hafta 22

Nerede kalmıştık?

2. haftada **ikili mod çalışması** (dual-mode operation) ile tanıştık ve `strace` ile ilk **sistem çağrısını** (system call) izledik: `printf()` sonunda `write(1, …)` oldu ve en küçük bir sistem çağrısı yaklaşık 116 ns, yani bir fonksiyon çağrısının kabaca 76 katı sürdü. Bu hafta önce sistem çağrısı kapısının ardında hangi hizmetlerin bulunduğunu (Kısım 1) ve bir isteğin bu kapıdan nasıl geçtiğini (Kısım 2) soruyoruz; ardından **sürece** (process) geçiyoruz (Kısım 3–7).

1 İşletim Sistemi Hizmetleri

İşletim sistemi, programların içinde çalıştığı bir ortam sağlar. Sunulan hizmetler işletim sisteminden işletim sistemine değişir, ama yaygın olanlar iki gruba ayrılır: *kullanıcıya* (ve kullanıcının programlarına) yardım eden hizmetler ve birçok kullanıcı ve program sistemi paylaşırken *sistemin kendisini* verimli ve güvenli biçimde çalışır tutan hizmetler.



Şekil 1: İşletim sistemi hizmetlerine genel bakış (Silberschatz vd., Şekil 2.1'den uyarlanmıştır). Nane yeşili: kullanıcıya yönelik hizmetler; gri: sistemin kendisine yönelik hizmetler.

1.1 Kullanıcıya yardım eden hizmetler

- **Kullanıcı arayüzü.** Neredeyse her işletim sisteminde bulunur. **Komut satırı arayüzü** (command-line interface, CLI) yazılan komutları okur; **grafik kullanıcı arayüzü** (graphical user interface, GUI) pencereler, simgeler ve bir işaretçi kullanır; telefonlar ve tabletler *dokunmatik ekran* arayüzü kullanır; *toplu iş* (batch) arayüzü ise komutları bir dosyadan okur.
- **Program yürütme.** İşletim sistemi bir programı belleğe yükler, çalıştırır ve sonlandırır: ya normal biçimde ya da olağan dışı biçimde (bir hata koduyla).
- **G/Ç (giriş/çıkış, I/O) işlemleri.** Çalışan bir program bir dosyaya ya da bir aygıta ihtiyaç duyabilir. İkili mod nedeniyle programlar aygıtları kendileri süremez; G/Ç'yi onlar adına işletim sistemi yapar.
- **Dosya sistemi işlemleri.** Programlar dosya ve dizin oluşturur, siler, okur, yazar, arar ve bunların izinlerini yönetir.
- **İletişim.** Süreçler aynı bilgisayarda ya da ağ üzerinden farklı bilgisayarlar arasında bilgi alışverişi yapar. Bu, **paylaşımlı bellek** (shared memory; iki süreç aynı bellek alanını okur ve yazar) ya da **mesaj geçirme** (message passing; işletim sistemi veri paketlerini süreçler arasında taşır, ör. borular ve soketler) ile yapılır.
- **Hata algılama.** Hatalar işlemcide ve bellekte, aygıtlarda (bir disk okuması başarısız olur, bir ağ bağlantısı kopar, yazıcının kâğıdı biter) ve kullanıcı programlarında (sıfıra bölme, yasak bir adrese erişim) ortaya çıkabilir. İşletim sistemi her hata türü için doğru eylemi seçmelidir: yeniden denemek, bildirmek ya da programı sonlandırmak.

1.2 Sistemin kendisi için hizmetler

- **Kaynak tahsisi.** Birden çok kullanıcı ya da iş aynı anda çalışırken her biri işlemciden, bellekten, dosyalardan ve aygıtlardan payını almalıdır. Bu işi işlemci zamanlayıcıları (4–5. haftalar) ve bellek ayırıcıları (6–7. haftalar) yapar. Linux'ta `ulimit -a`, işletim sisteminin uyguladığı süreç başına sınırları gösterir.
- **Hesap tutma ve günlük kaydı.** İşletim sistemi hangi kullanıcıların ve programların hangi kaynaktan ne kadar kullandığını faturalama, istatistik ve ayar için izler. Linux'ta `ps`, `top` ve sistem günlüğü (`journalctl`), Windows'ta Olay Görüntüleyicisi bu bilgiyi gösterir.
- **Koruma ve güvenlik.** *Koruma* (protection), süreçlerin kaynaklara erişimini denetlemek demektir: bir süreç başka bir sürecin belleğini okuyamamalı ve üzerine yazamamalıdır. *Güvenlik* (security), sistemi dışarıdan ve içeriden gelen saldırılara karşı savunmak demektir: kullanıcılar kimliklerini doğrulamalıdır (genellikle bir parolayla) ve izinsiz giriş denemeleri kaydedilir.

Zincir en zayıf halkası kadar güçlüdür: bir saldırgan yöneticinin parolasını tahmin edebiliyorsa 2. haftadaki bütün koruma donanımı bir işe yaramaz.

1.3 Kullanıcı arayüzleri: komut satırı ve GUI

Yazılan komutları okuyup yürüten programa **komut yorumlayıcısı** (command interpreter) ya da **kabuk** (shell) denir. Linux ve macOS'ta yaygın kabuklar `bash` ve `zsh`, Windows'ta `cmd.exe` ve PowerShell'dir. İki nokta önemlidir:

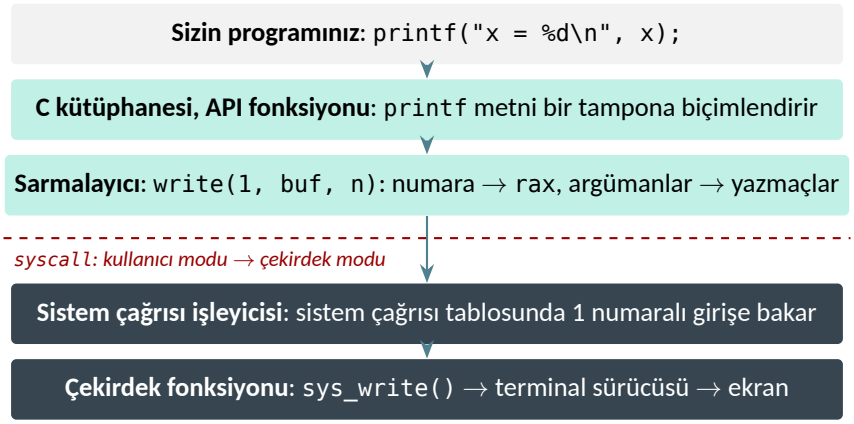
1. Kabuk çekirdeğin parçası *değildir*. Kullanıcı modunda çalışan sıradan bir sistem programıdır ve diğer her program gibi tam olarak aynı sistem çağrılarını kullanır.
2. Komutların çoğu kendi başına ayrı programlardır. `ls` yazdığınızda kabuk program dosyasını (`/usr/bin/ls`) bulur ve işletim sisteminden onu yeni bir süreç olarak çalıştırmasını ister (nasıl olduğunu Kısım 7 gösterir).

Komut satırı deneyimli kullanıcılar için hızlıdır ve betiklerle kolayca otomatikleştirilir; grafik arayüzün öğrenilmesi daha kolaydır. 9. haftadan itibaren Linux kabuğunda çalışacağız.

2 Ayrıntılı Sistem Çağrıları

2.1 API, kütüphane sarmalayıcısı ve sistem çağrısı

Programcılar neredeyse hiçbir zaman bir sistem çağrısını doğrudan yürütmez. `printf("x = %d\n", x);` gibi bir satır ile donanım arasında birkaç katman vardır.



Şekil 2: `printf()`'ten ekrana. Kesikli çizginin üstü: kullanıcı modu; altı: çekirdek modu.

Üç sözcük kolayca karıştırılır:

- **API** (application programming interface, uygulama programlama arayüzü), bir programcının çağırabileceği, parametreleri ve dönüş değerleriyle birlikte belgelenmiş fonksiyonlar kümesidir. Örnekler: *POSIX* API (Linux, macOS, BSD'ler), *Windows API* ve *Java API*.
- **Sarmalayıcı** (wrapper), genellikle sistem çağrısıyla aynı adı taşıyan (`write`, `read`, `fork`) minik bir kütüphane fonksiyonudur; tek işi argümanları çekirdeğin beklediği yere koymak ve tuzak (trap) komutunu yürütmektir.
- **Sistem çağrısı** ise çekirdeğe yapılan asıl, denetimli giriştir.

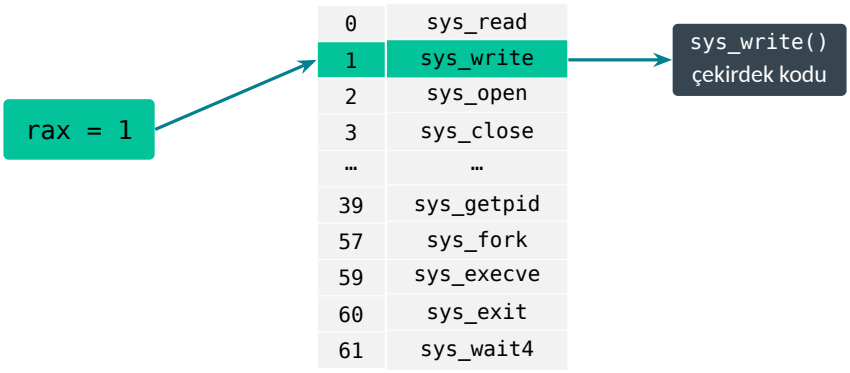
Her API fonksiyonu bir sistem çağrısı yapmaz. `strlen()` ve `sqrt()` hiçbir zaman yapmaz: sonuçlarını kullanıcı modunda hesaplarlar. `printf()` yalnızca tamponu boşaltıldığında sistem çağrısı yapar; bu yüzden on `printf()` çağrısı tek bir `write` üretebilir. Linux'ta C kütüphanesi genellikle *glibc*'dir; Windows'ta `kernel32.dll` içindeki Windows API fonksiyonları, asıl çekirdek çağrılarını yürüten `ntdll.dll` içindeki yerel arayüzü çağırır (ör. `CreateProcess()` sonunda `NtCreateUserProcess`'i çağırır).

2.2 Neden bir API'ye göre programlanır?

Asıl neden **taşınabilirliktir** (portability). POSIX API'sine göre yazılmış bir program her POSIX sisteminde derlenir; oysa sistem çağrısı *numaraları* ve ayrıntıları işletim sistemleri arasında, hatta aynı işletim sisteminin farklı işlemci mimarileri arasında bile değişir: `write`, 64 bit x86 Linux'ta 1, 32 bit x86 Linux'ta 4, 64 bit ARM Linux'ta 64 numaralı sistem çağrısıdır; API adı `write()` ise üçünde de aynıdır. İkinci neden **kolaylıktır**: API fonksiyonlarını kullanmak daha kolaydır ve çoğu zaman `printf()`'in biçimlendirme ve tamponlaması gibi yararlı ek işler yaparlar.

2.3 Sistem çağrısı numarası ve sistem çağrısı tablosu

Her sistem çağrısı küçük bir tamsayıyla, **sistem çağrısı numarasıyla** tanımlanır. Çekirdek bir **sistem çağrısı tablosu** (system-call table) tutar: bu numarayla indekslenen ve işi yapan çekirdek fonksiyonlarına işaret eden işaretçilerden oluşan bir dizi.



Şekil 3: `rax` içindeki numara, sistem çağrısı tablosunun bir girişini seçer. Numaralar gerçek x86-64 Linux numaralarıdır.

Neden bir tablo da doğrudan `sys_write`'ın adresine bir atlama değil? Çünkü bir kullanıcı programının rastgele bir çekirdek adresine atlamasına *asla* izin verilmemelidir: böylece bir çekirdek fonksiyonunun başındaki izin denetimlerini atlayabilirdi. Bu yüzden tuzak komutu çekirdeğe her zaman *tek* bir sabit giriş noktasından girer. Çekirdek numaranın geçerli olup olmadığını denetler (geçersiz bir numara `ENOSYS` hatasını döndürür) ve ancak ondan sonra tablodaki fonksiyonu çağırır. Bunu 2. haftadaki kesme vektörüyle karşılaştırın: ikisi de bir numarayla indekslenen işleyici adresleri tablosudur. Test sistemimizde (çekirdek 6.8 için Linux başlık dosyaları, x86-64) 0'dan 461'e numaralanmış yaklaşık 370 sistem çağrısı tanımlıdır.

2.4 Adım adım tuzak

64 bit x86 Linux'ta bir program `write(1, msg, 26)` çağırdığında olanlar şunlardır:

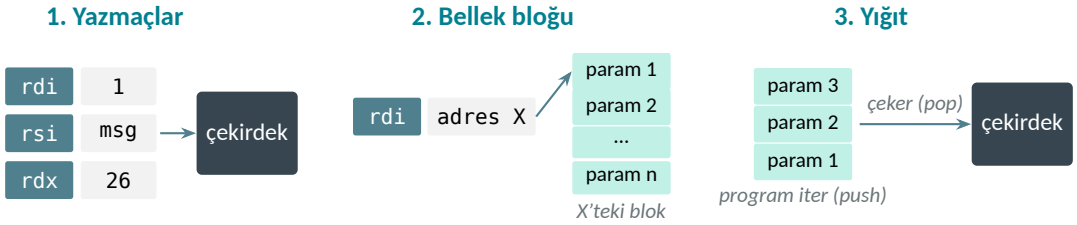
1. **Yazmaçları doldur** (kullanıcı modu). C kütüphanesindeki `write()` sarmalayıcısı sistem çağrısı numarası 1'i `rax` yazmacına, üç parametreyi de `rdi`, `rsi` ve `rdx` yazmaçlarına koyar.
2. **Tuzak.** Sarmalayıcı `syscall` komutunu yürütür. İşlemci mod bitini çekirdek moduna çevirir, dönüş adresini ve bayrakları kaydeder ve çekirdeğin tek sistem çağrısı giriş noktasına atlar.
3. **Kaydet ve denetle** (çekirdek modu). Çekirdek kullanıcının yazmaçlarını kaydeder ve 1'in geçerli bir sistem çağrısı numarası olduğunu denetler.

4. **Yönlendirme** (dispatch). Çekirdek tablo üzerinden `sys_write()`'ı çağırır. `sys_write()`, 1 numaralı dosya tanımlayıcısının açık olduğunu ve tampon adresinin gerçekten çağıran sürece ait olduğunu denetler, sonra baytları terminal sürücüsüne iletir.
5. **Sonuç.** Dönüş değeri, yani yazılan bayt sayısı (26) ya da negatif bir hata kodu `rax`'e konur.
6. **Dönüş.** `sysret` komutu kullanıcı durumunu geri yükler ve kullanıcı moduna geri döner. Sarmalayıcı programa 26 döndürür.

4. adımdaki denetim önemlidir: o olmasaydı bir program, çekirdekten bir çekirdek adresindeki baytları ekrana "yazmasını" isteyerek çekirdeğin sırlarını okuyabilirdi. Sistem çağrısı bir istektir ve çekirdek bu istekteki hiçbir şeye güvenmez.

2.5 Parametre aktarmanın üç yolu

Bir sistem çağrısı genellikle numarasından daha fazla bilgiye ihtiyaç duyar: hangi dosya, hangi tampon, kaç bayt. Bu parametreleri işletim sistemine aktarmanın üç genel yöntemi vardır.



Şekil 4: Bir sistem çağrısına parametre aktarmanın üç yolu.

1. **Yazmaçlarda.** En hızlı yöntemdir: hiçbir bellek erişimi gerekmez. Sınırı, kullanılabilir yazmaç sayısıdır; x86-64 Linux altı parametreye izin verir.
2. **Bellekte bir blokta (tabloda).** Program parametreleri bir bellek bloğuna yazar ve bir yazmaçta yalnızca bloğun *adresini* aktarır. İstenen sayıda parametre mümkündür. Linux bunu çok seçenekli sistem çağrıları için kullanır; örneğin `clone3()` bir `struct clone_args` yapısına işaretçi alır.
3. **Yığıtta** (stack). Program parametreleri yığıtına *iter* (push); işletim sistemi onları oradan *çeker* (pop, yani okur). Bu yöntemi örneğin x86 üzerinde 32 bit FreeBSD kullanıyordu.

2. ve 3. yöntemlerin gizli bir maliyeti vardır: çekirdek parametreleri kullanıcı belleğinden çekirdek belleğine kopyalamalı ve adreslerin gerçekten çağıran sürece ait olduğunu denetlemelidir. Linux 1. ve 2. yöntemleri birleştirir: altı parametreye kadar yazmaçlarda, daha büyük her şey (bir tampon, bir yapı) bir yazmaçta adres olarak aktarılır. Dikkat edin: 1. yöntemde bile mesajın 26 baytı kullanıcı belleğinde kalır; yazmaçta yalnızca adresleri taşınır.

2.6 Somut örnek: x86-64 Linux kuralı

Yazmaç	Sistem çağrısındaki rolü	`write(1, msg, 26)` için
`rax`	sistem çağrısı numarası (giriş), dönüş değeri (çıkış)	girişte 1, çıkışta 26 (yazılan bayt)
`rdi`	1. parametre	1 (dosya tanımlayıcısı: standart çıktı)
`rsi`	2. parametre	`msg`'nin adresi
`rdx`	3. parametre	26 (bayt sayısı)
`r10`, `r8`, `r9`	4., 5., 6. parametre	`write` kullanmaz
`rcx`, `r11`	`syscall` komutu bunların üzerine yazar	işlemci dönüş adresini ve bayrakları buraya kaydeder

x86-64'te sıradan C fonksiyon çağrıları dördüncü argümanı `rcx`'te aktarır; sistem çağrısı kuralı ise onun yerine `r10`'u kullanır, çünkü `syscall` komutunun kendisi `rcx`'in üzerine yazar. -4095 ile -1 arasındaki bir dönüş değeri hata demektir: sarmalayıcı pozitif hata numarasını `errno` global değişkenine yazar ve programa -1 döndürür. Yazmaçları ezberlemeniz gerekmez; önemli olan, parametrelerin yazmaçlarda taşındığı fikridir.

2.7 Örnek 1: aynı mesaj, üç düzey derinlikte

Bu kısımdaki programlar, bu notların yanındaki `code/` klasöründe bir `Makefile` ile birlikte bulunur (`make` yazın). Linux için C ile yazılmışlardır; 2. haftada olduğu gibi sizden C yazmanız beklenmez, yalnızca açıklamaları okuyup ne olduğunu anlamanız beklenir. İlk program üç mesaj yazdırır: `printf()` ile, `write()` sarmalayıcısı ile ve `syscall` komutunu, sarmalayıcının yaptığını aynen yapan birkaç satırlık satır içi assembly ile elle yürüterek.

```c
/* raw_write.c -- tek mesaj, üç düzey: kütüphane, sarmalayıcı, çıplak tuzak.
 * Bir sistem çağrısının bir NUMARA ile seçildiğini ve parametrelerinin
 * işlemci yazmaçlarında taşındığını gösterir (yalnızca x86-64 Linux).
 * Derleme:  gcc -Wall -O2 raw_write.c -o raw_write
 * İzle:     strace ./raw_write
 */
#include <stdio.h>    /* printf: C kütüphanesi (kullanıcı modu) */
#include <unistd.h>   /* write : ince libc sarmalayıcısı        */

int main(void)
{
    printf("1) printf  -> libc -> write\n");     /* düzey 1 */
    fflush(stdout);

    /* düzey 2 */
    if (write(1, "2) write() -> libc wrapper\n", 27) != 27)
        return 1;

    /* düzey 3: sarmalayıcının yaptığını elle yapıyoruz.
     *   rax = 1   write'ın sistem çağrısı numarası (x86-64)
     *   rdi = 1   1. parametre: dosya tanımlayıcısı (stdout)
     *   rsi = msg 2. parametre: baytların adresi
     *   rdx = 26  3. parametre: bayt sayısı
```

```
     * 'syscall' komutu çekirdeğe tuzak kurar; çekirdeğin
     * dönüş değeri rax'te geri gelir.                    */
    static const char msg[] = "3) syscall instruction!!!\n";
    long ret;
    __asm__ volatile ("syscall"
                      : "=a"(ret)               /* çıkış: rax */
                      : "a"(1L), "D"(1L), "S"(msg), "d"(26L)
                      : "rcx", "r11", "memory"); /* bozulanlar */
    printf("kernel returned %ld (bytes written)\n", ret);
    return 0;
}
```

```
$ make raw_write && strace ./raw_write
...
write(1, "1) printf  -> libc -> write\n", 28) = 28
write(1, "2) write() -> libc wrapper\n", 27) = 27
write(1, "3) syscall instruction!!!\n", 26) = 26
write(1, "kernel returned 26 (bytes writte"..., 35) = 35
exit_group(0)                           = ?
```

İki gözlem. Birincisi, `strace` üç düzeyi birbirinden ayıramaz: çekirdek için üçü de üç parametreli aynı 1 numaralı sistem çağrısıdır. İkincisi, çekirdek `rax`'te 26 döndürdü; bu, yazmasını istediğimiz bayt sayısının ta kendisidir. Satır içi assembly yalnızca gösterim içindir ve yalnızca x86-64 Linux'ta çalışır; ARM işlemcilerde tuzak komutu `svc`'dir ve farklı yazmaçlar kullanılır.

2.8 Sistem çağrılarının altı kategorisi

Sistem çağrıları altı kategoride toplanabilir. Tablo tipik örnekler veriyor; Windows sütunu, her biri bir ya da daha fazla yerel sistem çağrısını çağıran Windows API fonksiyonlarını listeler.

Kategori	Tipik istekler	Linux (POSIX)	Windows API
Süreç denetimi	oluştur, sonlandır, yükle, bekle	`fork()`, `execve()`, `exit()`, `wait4()`	`CreateProcess()`, `ExitProcess()`, `WaitForSingleObject()`
Dosya yönetimi	oluştur, aç, oku, yaz, kapat	`open()`, `read()`, `write()`, `close()`	`CreateFile()`, `ReadFile()`, `WriteFile()`, `CloseHandle()`
Aygıt yönetimi	iste, oku, yaz, yapılandır	`ioctl()`, `read()`, `write()`	`SetConsoleMode()`, `ReadConsole()`, `WriteConsole()`
Bilgi bakımı	saat, tarih, kimlikler, sistem verisi	`getpid()`, `uname()`, `clock_gettime()`	`GetCurrentProcessId()`, `GetSystemTime()`
İletişim	boru, paylaşımlı bellek, ağ	`pipe()`, `mmap()`, `socket()`	`CreatePipe()`, `MapViewOfFile()`
Koruma	izinler, sahiplik	`chmod()`, `chown()`, `umask()`	`SetFileSecurity()`, `SetSecurityDescriptorGroup()`

`read()` ve `write()`'ın hem dosya hem aygıt yönetiminde göründüğüne dikkat edin. UNIX'te bir aygıt özel bir dosya gibi görünür (örneğin terminal için `/dev/tty`); bu yüzden aynı sistem çağrıları ikisine de hizmet eder. Bu, 2. haftadaki genişletilmiş makine bakışının iş başındaki hâlidir: tek bir soyutlama, birçok donanım türü.

Bir dosyayı kopyalamak kaç sistem çağrısı gerektirir?

Bir dosyayı başka bir dosyaya kopyalayan bir program, klasik ders kitabı anlatımında şunlara ihtiyaç duyar:

1. İki dosya adını istemek (G/Ç sistem çağrıları ya da adları komut satırından okumak).
2. Girdi dosyasını açmak; dosya yoksa bir hata mesajı yazdırıp çıkmak (dosya yönetimi, süreç denetimi).
3. Çıktı dosyasını oluşturmak.
4. Döngü: `read()` 0 döndürene (dosya sonu) kadar girdiden bir blok okuyup çıktıya yazmak.
5. İki dosyayı da kapatmak, başarıyı bildirmek ve çıkmak.

B baytlık bir tampon ve S baytlık bir dosyayla yalnızca döngü, veri döndüren $\lceil S/B \rceil$ okuma, 0 döndüren bir son okuma ve $\lceil S/B \rceil$ yazma yapar. $S = 1$ MiB $= 2^{20}$ bayt ve $B = 4096 = 2^{12}$ bayt için bu, $2 \cdot 2^8 + 1 = 513$ sistem çağrısı eder.
Test makinemizdeki gerçek GNU `cp` (sürüm 9.4) farklı davranır. 1 MiB'lık bir dosya için `strace -c cp in.bin out.bin` toplam 102 sistem çağrısı saydı; en büyük gruplar `mmap` (26), `close` (13), `openat` (11), `fstat` (10) ve `read` (9) idi — `copy_file_range` ise yalnızca 2 kez çağrıldı. 102 çağrıdan yalnızca 2'si veri taşır: `copy_file_range`, çekirdekten bütün dosyayı baytları programın belleğinden geçirmeden *çekirdeğin içinde* kopyalamasını ister. Geri kalan çağrıların çoğu programı ve paylaşımlı kütüphanelerini yükler (`mmap`, `openat`, kütüphane başlıklarının `read` ile okunması). Sistem çağrısından tasarruf etmek için işi çekirdeğe taşımak, işletim sistemi tasarımında sık rastlanan bir temadır.

2.9 Sistem programları

Sistem programları (system programs; sistem araçları da denir) işletim sistemiyle birlikte gelir ve program geliştirme ile yürütme için elverişli bir ortam sağlar. Kullanıcı modunda çalışırlar ve sizin adınıza sistem çağrısı yaparlar. Kullanıcıların çoğu işletim sistemini sistem çağrıları üzerinden değil, bu programlar üzerinden görür. Başlıca kategoriler şunlardır: **dosya yönetimi** (`ls`, `cp`, `rm`, Dosya Gezgini); **durum bilgisi** (`date`, `df`, `ps`, `top`, Görev Yöneticisi, sistem günlükleri); **dosya düzenleme** (`nano` ve Not Defteri gibi düzenleyiciler, `grep` gibi arama araçları); **programlama dili desteği** (derleyiciler, hata ayıklayıcılar, yorumlayıcılar: `gcc`, `gdb`, `python`); **program yükleme ve yürütme** (yükleyiciler ve bağlayıcılar); **iletişim** (`ssh`, web tarayıcıları, e-posta); ve açılışta başlayıp sürekli çalışan **arka plan hizmetleri**: UNIX'te **daemon**, Windows'ta *service* denir (`sshd`, `cron`, yazdırma kuyruğu).

Bu programların çoğunu 9–14. haftalarda uygulamalı olarak kullanacağız.

3 Süreç Kavramı

3.1 Program ve süreç

Program *edilgen* bir varlıktır: diskte bir komut listesi içeren bir dosya, örneğin `/usr/bin/bash`. **Süreç** (process) ise *etkin* bir varlıktır: yürütülmekte olan bir program. Bir sürecin program kodunun yanı sıra şunları vardır:

- bir *o anki etkinlik*: **program sayacının** (program counter; bir sonraki komutun adresi) ve diğer işlemci

yazmaçlarının değeri;

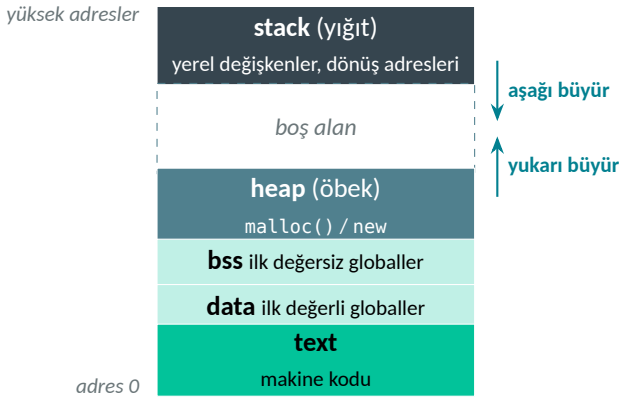
- kendine ait *bellek*: bir yığıt, bir öbek ve global veriler (Kısım 3.2);
- işletim sisteminin ona verdiği kaynaklar: açık dosyalar, ağ bağlantıları vb.

Bir program belleğe yüklendiğinde sürece dönüşür; örneğin bir simgeye çift tıkladığınızda ya da adını bir kabukta yazdığınızda. Tek bir program birçok sürecin temeli olabilir: iki terminal penceresi açarsanız aynı program dosyasından, her biri kendi belleği ve kendi ilerlemesiyle iki `bash` süreci çalışır. Eski kitaplar aynı fikir için *iş* (job) sözcüğünü kullanır (toplu işlem sistemlerinden kalma).

Benzetme: yemek tarifi bir programdır; onu pişirmek bir süreçtir. İki aşçı aynı tarifi aynı anda, her biri kendi malzemesi ve tarifte kendi bulunduğu yerle uygulayabilir.

3.2 Bellekteki bir süreç

Bir sürecin belleği bölgelere (segment ya da kesim de denir) ayrılır:



Şekil 5: Bir sürecin bellekteki düzeni.

- **Text:** programın derlenmiş makine kodu. Bir hata kodun üzerine yazamasın diye salt okunurdur; aynı programı çalıştıran birden çok süreç onu paylaşabilir.
- **Data:** ilk değeri olan global (ve `static`) değişkenler, ör. `int initialised = 42;`. Değerler çalıştırılabilir dosyada saklanır.
- **BSS:** ilk değeri *olmayan* global değişkenler, ör. `int uninitialised;`. İşletim sistemi program yüklenirken bu bölgeyi sıfırlarla doldurur; bu yüzden çalıştırılabilir dosyanın yalnızca boyutunu saklaması yeterlidir. (Ad tarihseldir: eski bir assembler'dan kalma "block started by symbol".)
- **Heap (öbek):** program çalışırken istenen bellek; C'de `malloc()`, C++ ve Java'da `new` ile. Açıkça serbest bırakılana kadar yaşar.
- **Stack (yığıt):** her etkin fonksiyon çağrısı için bir *çerçeve* (frame); yerel değişkenleri, parametreleri ve dönüş adresini tutar. Fonksiyon döndüğünde çerçeve kaldırılır.

Text, data ve bss program yüklendikten sonra sabit boyutludur; öbek ve yığıt ise program çalışırken büyüyüp küçülür, aralarındaki boş alanda birbirlerine doğru ilerler. Çok fazla büyüyen bir yığıt, örneğin sonsuz özyineleme yüzünden, bir *yığıt taşmasına* (stack overflow) yol açar ve program sonlandırılır.

3.3 Örnek 2: her bölgenin adresini yazdırmak

```c
/* mem_layout.c -- bir sürecin parçaları bellekte nerede durur?
 * Süreç görüntüsünün her bölgesinden birer adres yazdırır.
 * Derleme:  gcc -Wall -O2 mem_layout.c -o mem_layout
 * Çalıştır: ./mem_layout  (iki kez çalıştırın: adresler değişir,
 *                          çünkü Linux onları rastgele yerleştirir: ASLR)
 */
#include <stdio.h>
#include <stdlib.h>

int initialised = 42;   /* değeri olan global   -> data */
int uninitialised;      /* değeri olmayan global -> bss  */

int main(void)
{
    int local = 7;                  /* yerel değişken -> yığıt */
    int *dynamic = malloc(sizeof *dynamic);     /* malloc -> öbek */

    printf("text  main()        %p\n", (void *)main);
    printf("data  initialised   %p\n", (void *)&initialised);
    printf("bss   uninitialised %p\n", (void *)&uninitialised);
    printf("heap  *dynamic      %p\n", (void *)dynamic);
    printf("stack local         %p\n", (void *)&local);

    free(dynamic);
    return 0;
}
```

```
$ ./mem_layout
text  main()        0x555d9ba160c0
data  initialised   0x555d9ba19010
bss   uninitialised 0x555d9ba19018
heap  *dynamic      0x555db649d2a0
stack local         0x7ffe2b4f6ab4
```

Adreslerin sırası şekle uyar: text $<$ data $<$ bss $<$ heap, yığıt ise çok daha yukarıda. İlk çalıştırmada data değişkeni `main`'in `0x555d9ba19010` $-$ `0x555d9ba160c0` $=$ `0x2f50` $= 12\,112$ bayt yukarısındadır; bss değişkeni ise data değişkeninin yalnızca 8 bayt yukarısındadır: yan yana dururlar. Öbek ile yığıt arasındaki boşluk `0x7ffe2b4f6ab4` $-$ `0x555db649d2a0` $\approx 4.7 \times 10^{13}$ bayt, yaklaşık 43 TiB'tır; bu, herhangi bir kişisel bilgisayarın RAM'inden çok daha fazladır. Bu mümkündür, çünkü bunlar *sanal* adreslerdir; yalnızca gerçekten kullanılan kısımların arkasında fiziksel bellek bulunur (7. hafta).

Programı yeniden çalıştırın; her adres farklı çıkar: örneğin ikinci çalıştırmamızda `main` `0x55a4811220c0`, `local` ise `0x7ffec440c1f4` adresindeydi. Linux, bir program her başladığında bölgeleri rastgele konumlara yerleştirir; bu tekniğe **adres alanı düzeninin rastgeleleştirilmesi** (address space layout randomisation, ASLR) denir. Böylece bir programda hata bulan bir saldırgan, programın kodunun ve verisinin nerede olduğunu önceden bilemez.

> **ASLR**
>
> ASLR, Linux için PaX güvenlik projesi tarafından 2001'de tanıtıldı ve 2005'te standart Linux çekirdeğine girdi; Windows onu Windows Vista ile (2007) benimsedi. Bugün masaüstü ve mobil sistemlerde standarttır.

4 Süreç Durumları

Bir süreç yürütülürken **durumu** (state) değişir. Her süreç beş durumdan birindedir:

- **Yeni** (new): süreç oluşturulmaktadır.
- **Hazır** (ready): süreç bir işlemciye atanmayı beklemektedir.
- **Çalışıyor** (running): komutları yürütülmektedir.
- **Bekliyor** (waiting; *engellenmiş*, blocked da denir): süreç bir G/Ç işleminin tamamlanması ya da bir çocuk sürecin bitmesi gibi bir olayı beklemektedir.
- **Sonlandı** (terminated): süreç yürütmesini bitirmiştir.



Şekil 6: Beş durumlu süreç modeli.

Bir işlemci çekirdeğinde herhangi bir anda yalnızca bir süreç *çalışıyor* olabilir; birçok süreç *hazır* ya da *bekliyor* durumunda olabilir. İşletim sistemi hazır süreçleri bir **hazır kuyruğunda** (ready queue), bekleyen süreçleri ise **bekleme kuyruklarında** (wait queues; her aygıt ya da olay için bir tane) tutar; her kuyruk süreç denetim bloklarından oluşan bir bağlı listedir (Kısım 5).

4.1 Her geçişi ne tetikler?

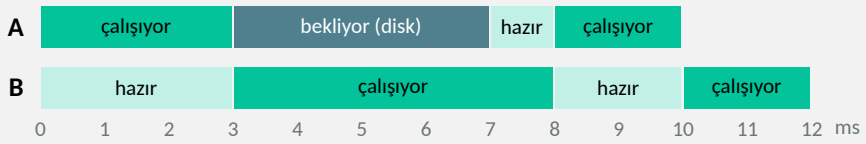
Geçiş	Tetikleyen	Gündelik örnek
yeni → hazır	işletim sistemi PCB'yi oluşturdu ve programı yükledi	`./mem_layout` yazıp Enter'a basarsınız
hazır → çalışıyor	zamanlayıcı süreci *görevlendirir* (dispatch)	bir işlemci çekirdeği boşalır ve zamanlayıcı onu seçer
çalışıyor → hazır	bir kesme, genellikle zamanlayıcı saat (zaman dilimi doldu)	başkaları çalışabilsin diye uzun bir hesap duraklatılır
çalışıyor → bekliyor	beklemesi gereken bir sistem çağrısı (G/Ç, çocuk, sleep)	diskten `read()`; bir çocuk için `wait()`
bekliyor → hazır	beklenen olay gerçekleşir; çoğunlukla bir kesmeyle bildirilir	disk kesmesi verinin geldiğini bildirir
çalışıyor → sonlandı	`exit()` ya da ölümcül bir hata	`main()` döner; program sıfıra böler

İki not. Birincisi, bir süreç asla *bekliyor* durumundan doğrudan *çalışıyor* durumuna geçemez: olayı gerçekleştiğinde hazır kuyruğuna katılır ve zamanlayıcı onu görevlendirene kadar bekler. Sırada hangi hazır sürecin çalışacağı *işlemci zamanlamasının* (CPU scheduling) konusudur (4–5. haftalar). İkincisi, geçişlere kimin yol açtığına bakın: bir süreç çalışıyor durumundan ya bir sistem çağrısı yaparak (Kısım 2) ya da bir kesme (2. hafta) işlemciyi elinden aldığı için çıkar. İşletim sistemi kesme güdümlüdür.

Linux'ta `ps` komutu durumu STAT sütununda gösterir: çalışıyor *ya da* hazır için `R` (Linux burada ikisini ayırmaz), uyuyor (kesilebilir bekleme) için `S`, kesilemez bekleme (genellikle disk G/Ç'si) için `D`, durdurulmuş için `T` ve zombi için `Z` (Kısım 7.4).

İki süreç, tek işlemci

A süreci 3 ms hesap yapar, diskten okur (4 ms), sonra 2 ms daha hesap yapar. B süreci 7 ms işlemci zamanına ihtiyaç duyar. İşletim sistemi her sürece en çok 5 ms'lik bir zaman dilimi verir. İkisi de $t = 0$'da hazırdır ve önce A görevlendirilir.



- $t = 3$: A `read()` çağırır ve *bekliyor* durumuna geçer; B görevlendirilir.
- $t = 7$: disk kesmesi A'yı *hazır* durumuna taşır; işlemci B'de kalır.
- $t = 8$: zamanlayıcı kesmesi B'nin 5 ms'lik dilimini bitirir; B hazır olur ve A görevlendirilir.
- $t = 10$: A çıkar; B kalan 2 ms'sini çalışır ve $t = 12$'de çıkar.

Gereken işlemci işi A için $3 + 2 = 5$ ms, B için 7 ms, toplam 12 ms'dir. Art arda çalıştırılsalar A $3 + 4 + 2 = 9$ ms sürer (işlemci 4 ms boşta kalır), B de 7 ms daha: toplam 16 ms; işlemci zamanın $12/16 = 75\%$'inde meşguldür. Yukarıdaki gibi iç içe çalıştırıldıklarında ikisi de 12 ms sonra biter ve

işlemci zamanın $12/12 = 100\%$'ünde meşguldür. Bu, 1. haftadaki çoklu programlama fikrinin süreç durumlarıyla anlatılmış hâlidir.

5 Süreç Denetim Bloğu

Her süreç işletim sisteminde bir **süreç denetim bloğu** (process control block, PCB) ile temsil edilir; buna *görev denetim bloğu* (task control block) da denir. PCB, çekirdek belleğinde bulunan ve işletim sisteminin süreç hakkında bilmesi gereken her şeyi tutan bir veri yapısıdır.

süreç durumu
süreç kimliği (PID), ebeveyn PID
program sayacı
işlemci yazmaçları
işlemci zamanlama bilgisi
bellek yönetimi bilgisi
hesap bilgisi
G/Ç durumu: açık dosyaların listesi, aygıtlar

Şekil 7: Bir süreç denetim bloğunun tipik içeriği.

- **Süreç durumu:** yeni, hazır, çalışıyor, bekliyor ya da sonlandı.
- **Kimlik:** süreç kimliği (PID), ebeveynin PID'si, sahip (kullanıcı kimliği).
- **Program sayacı:** yürütülecek bir sonraki komutun adresi.
- **İşlemci yazmaçları:** süreç çalışmayı bıraktığında değerleri kaydedilmelidir ki daha sonra doğru biçimde devam edebilsin.
- **İşlemci zamanlama bilgisi:** öncelik, zamanlama kuyruklarına işaretçiler.
- **Bellek yönetimi bilgisi:** sürecin bölgelerinin nerede olduğu, ör. taban ve sınır yazmaçları ya da sayfa tabloları (6–7. haftalar).
- **Hesap bilgisi:** kullanılan işlemci zamanı, zaman sınırları, bağlam değişimi sayısı.
- **G/Ç durum bilgisi:** açık dosyaların ve ayrılmış aygıtların listesi.

Bir süreç çalışmıyorken, işlemci açısından süreç PCB'sinin *ta kendisidir*: onu sürdürmek için gereken her şeyi içerir. Linux'ta PCB, çekirdek kaynak dosyası `include/linux/sched.h` içinde tanımlanan `struct task_struct` adlı bir C yapısıdır; yüzlerce alanı vardır.

5.1 Bir PCB'ye göz atmak: `/proc/<pid>/status`

Linux her PCB'nin bazı kısımlarını `/proc` dizininde metin dosyaları olarak görünür kılar. `/proc` diskte gerçek bir dizin değildir: siz okudukça çekirdek metni anında üretir. Test makinemizdeki bir `sleep` süreci için (`grep` deseni burada iki satıra bölündü; tek satırda yazın):

```
$ sleep 30 &
$ grep -E "^(Name|State|Pid|PPid|Uid|VmSize|VmRSS|Threads|voluntary)" \
```

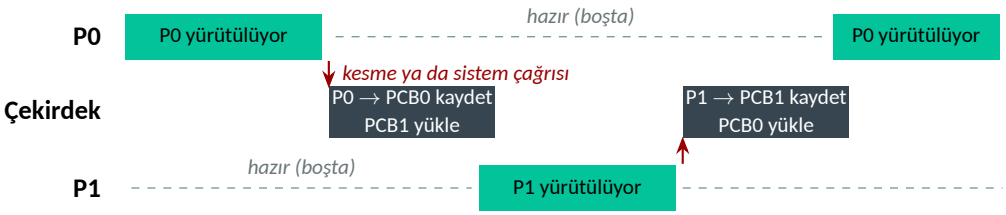
```
       /proc/$!/status
Name:   sleep
State:  S (sleeping)
Pid:    27271
PPid:   27265
Uid:    0       0       0       0
VmSize:     2724 kB
VmRSS:      1696 kB
Threads:        1
voluntary_ctxt_switches:        2
nonvoluntary_ctxt_switches:     0
```

`$!` en son arka plan komutunun PID'sidir. Alanlar doğrudan PCB'ye karşılık gelir: `State` süreç durumudur (`sleep` bir zamanlayıcıyı beklediği için S); `Pid` ve `PPid` süreci ve ebeveynini tanımlar; `Uid` sahiptir (burada kullanıcı 0, yani yönetici `root`); `VmSize` (sanal bellek boyutu) ve `VmRSS` (gerçekten RAM'de olan kısım) bellek yönetimi bilgisidir; iki bağlam değişimi sayacı da hesap bilgisidir. *Gönüllü* (voluntary) geçişler süreç bekleyerek işlemciyi bıraktığında, *gönülsüz* (non-voluntary) geçişler ise işletim sistemi işlemciyi elinden aldığında olur. `Threads` satırı gelecek haftanın konusudur.

6 Bağlam Değişimi

Bir kesme ya da bir sistem çağrısı işletim sistemini işlemciyi bir süreçten alıp başka birine vermeye yönelttiğinde, işletim sistemi bir **bağlam değişimi** (context switch) yapar. Bir sürecin *bağlamı* PCB'sinde temsil edilir: program sayacı, işlemci yazmaçları, yığıt işaretçisi, süreç durumu ve bellek yönetimi bilgisi (hangi sayfa tablosunun etkin olduğu). Bağlam değişimi iki adımdan oluşur:

1. **Durumu kaydetme** (state save): o anki sürecin bağlamını PCB'sine yazmak.
2. **Durumu geri yükleme** (state restore): bir sonraki sürecin bağlamını PCB'sinden yüklemek.



Şekil 8: P0'dan P1'e ve geri bağlam değişimi (Silberschatz vd., Şekil 3.6'dan uyarlanmıştır; zaman soldan sağa akacak biçimde çizildi).

Geçiş sırasında (koyu kutular) *hiçbir süreç yararlı iş yapmaz*: bağlam değişimi saf **ek yüktür** (overhead). Modern donanımda doğrudan maliyeti birkaç mikrosaniyedir; işlemci hızına, kopyalanacak yazmaç sayısına ve bellek yönetimi işine bağlıdır. Gizli, dolaylı bir maliyet de vardır: bir geçişten sonra işlemci önbellekleri ve TLB (6. hafta) önceki sürecin verileriyle doludur; bu yüzden yeni süreç bir süre daha yavaş çalışır.

Bağlam değişimi ne kadara mal olur?

Her sürecin, işletim sistemi geçiş yapmadan önce q zaman dilimi kadar çalıştığını ve her geçişin s kadar sürdüğünü varsayalım. Her $q + s$ dönemin yalnızca q kadarı yararlıdır; dolayısıyla kaybedilen işlemci

zamanı oranı

$$\text{ek yük} = \frac{s}{q + s}$$

olur. $s = 5$ μs alalım (temsilî bir değer):

Zaman dilimi q	Ek yük $s/(q+s)$	Yararlı iş
100 ms	0.005%	99.995%
10 ms	0.05%	99.95%
1 ms	0.50%	99.50%
0.1 ms	4.76%	95.24%
0.01 ms	33.3%	66.7%
0.005 ms ($q = s$)	50%	50%

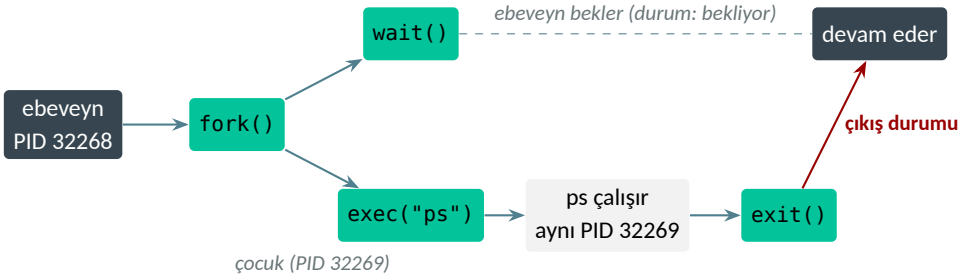
Örneğin $q = 1$ ms $= 1000$ μs ile: $5/(1000 + 5) = 0.004975 \approx 0.50\%$. Aynı şeyi başka biçimde söylersek, saniyede 1000 geçiş her saniyenin 1000×5 μs $= 5$ ms'sine, yani %0,5'ine mal olur.
Ders: kısa zaman dilimleri sistemin hızlı tepki veriyormuş gibi hissettirmesini sağlar ama işlemci zamanını geçişlerle harcar; uzun zaman dilimleri az harcar ama etkileşimli programları hantal hissettirir. q'yu seçmek, 5. haftadaki Round-Robin zamanlamasının temel sorusudur.

7 Süreç Oluşturma ve Sonlandırma

7.1 fork(), exec() ve wait()

Bir süreç yeni süreçler oluşturabilir. Oluşturan sürece **ebeveyn süreç** (parent process), yeni süreçlere de onun **çocuk süreçleri** (child processes) denir. UNIX ve Linux'ta işi üç sistem çağrısı yapar:

- `fork()`, çağıranın neredeyse tıpatıp bir *kopyası* olan yeni bir süreç oluşturur: aynı program, aynı bellek içeriği, aynı açık dosyalar. Tek fark dönüş değeridir: `fork()` çocukta 0, ebeveynde çocuğun PID'sini döndürür (hiçbir süreç oluşturulamadıysa -1). Bir kez çağrılır, iki kez döner.
- `exec()` (Linux'ta `execve()` sistem çağrısı; `execlp()` gibi kütüphane çeşitleri de vardır), çağıran süreçte çalışan programı yeni bir programla *değiştirir*. PID aynı kalır; eski kod, veri, öbek ve yığıt atılır. Başarılı olursa `exec()` asla geri dönmez.
- `wait()` (ya da `waitpid()`), ebeveyni bir çocuk sonlanana kadar *bekliyor* durumuna sokar ve sonra çocuğun çıkış durumunu toplar.



Şekil 9: `fork()`, `exec()` ve `wait()` ile süreç oluşturma. PID'ler aşağıdaki gerçek çalıştırmadan alındı.

Kabuğun yazdığınız her komut için yaptığı tam olarak budur: kendisinin bir kopyasını fork eder, kopya (çocuk) komutu exec eder ve kabuk bir sonraki komut istemini yazdırmadan önce onun bitmesini bekler. Neden iki ayrı adım? `fork()` ile `exec()` arasında çocuk kendi ortamını hazırlayabilir, örneğin çıktısını bir dosyaya yönlendirebilir. Bunu 12. haftada (borular ve yönlendirme) kullanacağız. Windows iki adımı tek bir çağrıda, çalıştırılacak programın adını parametre olarak alan `CreateProcess()` çağrısında birleştirir.

7.2 Örnek 3: C ile fork, exec ve wait

Listeleme `code/fork_exec_wait.c` dosyasının özünü gösteriyor; tam dosya buna `#include` satırlarını ve başarısız bir `fork()` (bu durumda -1 döner) için bir hata denetimini ekler.

```c
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);           /* fork'tan ÖNCE tamponu boşalt */

    pid_t pid = fork();       /* biri çağırır, ikisi döner */

    if (pid == 0) {           /* ---- çocuk: fork 0 döndürdü */
        printf("child : PID %d, my parent is %d\n",
               getpid(), getppid());
        fflush(stdout);
        char list[32];        /* ps -p için "ebeveyn,çocuk" */
        snprintf(list, sizeof list, "%d,%d", getppid(), getpid());
        execlp("ps", "ps", "-o", "pid,ppid,stat,comm",
               "-p", list, NULL);   /* bu programın yerine geçer */
        exit(127);            /* yalnızca exec başarısızsa */
    }
    int status;               /* ---- ebeveyn */
    waitpid(pid, &status, 0); /* çocuk bitene kadar uyu */
    printf("parent: child %d finished, exit status %d\n",
           pid, WEXITSTATUS(status));
}
/* hata denetimli tam program: code/fork_exec_wait.c */
```

```
$ ./fork_exec_wait
parent: my PID is 32268
child : PID 32269, my parent is 32268
  PID  PPID STAT COMMAND
32268 32251 S    fork_exec_wait
32269 32268 R    ps
parent: child 32269 finished, exit status 0
```

Aynı `if` testi iki farklı yanıt verir: çocuk `pid == 0` görür, ebeveyn çocuğun PID'sini. `exec()`'ten sonra çocuğun yazdırdığı `ps` çıktısı PID 32269'u `ps` olarak listeler: süreç PID'sini korudu, ama artık farklı bir program çalıştırıyor. Durumu R'dir (çalışıyor: listelemeyi yapan süreç odur); ebeveyn ise `waitpid()` içinde beklediği için S'dir.

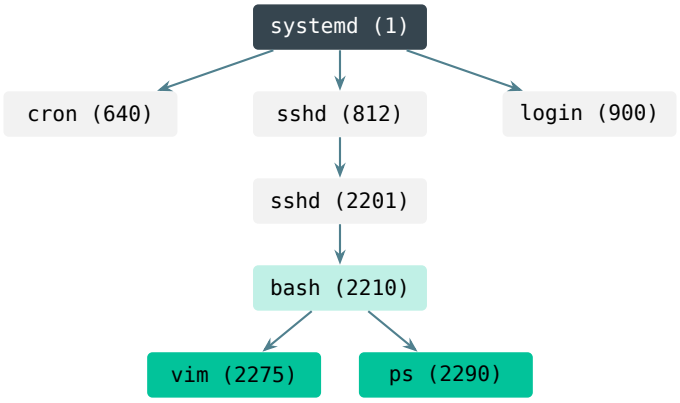
`strace -f` altında üç çağrı `clone(…) = 6411`, `execve("/usr/bin/ps", …) = 0` ve `wait4(6411, …) = 6411` olarak görünür (başka bir çalıştırmadan): C kütüphanesi `fork()`'u daha genel Linux sistem çağrısı `clone()` ile gerçekler.

`fork()`'tan önce neden `fflush(stdout)`?

`printf()` metni sürecin belleğindeki bir tamponda toplar. `fork()` bu belleği, henüz yazılmamış metin dahil, kopyalar. İlk `fflush(stdout)` satırını silin, `./fork_exec_wait > out.txt` çalıştırın (dosyaya giden çıktı tamamen tamponlanır) ve `out.txt`'ye bakın: "parent: my PID is …" satırı her süreç tarafından birer kez olmak üzere iki kez görünür.

7.3 Süreç ağacı ve süreç kimlikleri

Her sürecin benzersiz bir tamsayı **süreç kimliği** (process identifier, PID) vardır ve süreç ebeveyninin PID'sini (PPID) bilir. İlki dışındaki her süreç başka bir süreç tarafından oluşturulduğu için süreçler bir **süreç ağacı** (process tree) oluşturur. Linux'ta kullanıcı alanı ağacının kökü PID 1'dir; adı `systemd`'dir (eski ve küçük sistemlerde `init`) ve çekirdek onu açılışın sonunda başlatır; çekirdek iş parçacıkları ayrı bir kökün, `kthreadd`'nin (PID 2) altında durur.



Şekil 10: Tipik bir Linux süreç ağacının bir parçası (PID'ler temsilî). SSH ile oturum açmış bir kullanıcı bir `bash` kabuğundan `vim` ve `ps` çalıştırıyor.

`pstree -p` komutu gerçek ağacı çizer. Test makinemizde:

```
$ bash -c 'sleep 5 & sleep 5 & pstree -p $$'
bash(6297)-+-pstree(6300)
           |-sleep(6298)
           `-sleep(6299)
```

`$$` kabuğun kendi PID'sidir. Yeni `bash` üç çocuk oluşturdu; hepsi ondan fork edildi. PID'ler artan sırayla atanır ve bir süreç bittikten sonra yeniden kullanılır; en büyük PID `/proc/sys/kernel/pid_max` dosyasında

ayarlanır (test makinemizde 32768).

7.4 Süreç sonlandırma, zombiler ve yetimler

Bir süreç `exit()` çağırdığında (`main()`'den dönmek bunu örtük olarak yapar) ya da işletim sistemi onu ölümcül bir hatadan sonra öldürdüğünde sonlanır. İşletim sistemi bunun üzerine belleğini serbest bırakır ve dosyalarını kapatır. Ama PID'yi ve *çıkış durumunu* tutan PCB girdisi, ebeveyn onu `wait()` ile toplayana kadar yaşamalıdır.

- **Zombi** (zombie), sonlanmış ama ebeveyni henüz `wait()` çağırmamış bir süreçtir. Bellek ya da işlemci kullanmaz, ama süreç tablosunda bir girdi işgal eder ve PID'sini tutar. Çok sayıda çocuk oluşturup hiçbirini beklemeyen bir ebeveyn süreç tablosunu doldurabilir; o zaman yeni süreç oluşturulamaz.
- **Yetim** (orphan), ebeveyni ondan önce sonlanmış bir süreçtir. Linux yetimleri PID 1'e (ya da belirlenmiş bir "subreaper" sürecine) *yeniden bağlar*; bu süreç düzenli olarak `wait()` çağırır, böylece yetimler uzun süre zombi kalmaz. Bazı sistemler ise bir ebeveyn bittiğinde tüm çocuklarını sonlandırır (*zincirleme sonlandırma*, cascading termination).

Bir kabuk numarasıyla zombi üretmek kolaydır: `sh -c 'sleep 1 & exec sleep 4'`, `sleep 1`'i arka planda başlatır ve sonra kabuğun *yerine* hiç `wait()` çağırmayan bir program olan `sleep 4`'ü koyar. İki saniye sonra test makinemizde:

```
$ ps -o pid,ppid,stat,cmd -p 6385 --ppid 6385
  PID  PPID STAT CMD
 6385  6384 S    sleep 4
 6387  6385 Z    [sleep] <defunct>
```

Biten `sleep 1`, `Z` durumunda gösterilir ve `<defunct>` olarak işaretlenir. İki saniye sonra `sleep 4` bittiğinde zombi yetim kalır, yeni ebeveyni tarafından evlat edinilir ve toplanır.

7.5 Örnek 4: n kez fork() çağrısından sonra kaç süreç olur?

`code/fork_count.c` programı n'yi komut satırından okur ve sonra şunu yürütür:

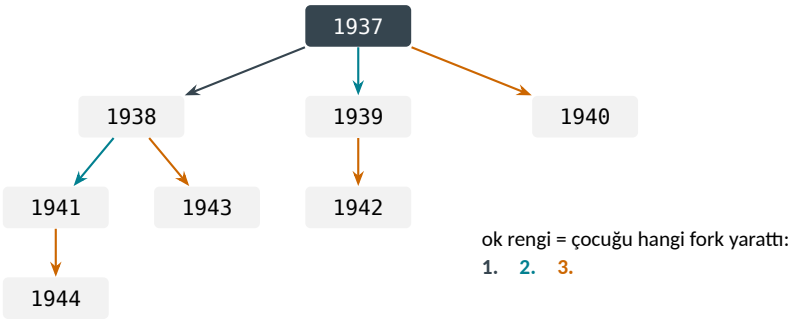
```c
for (int i = 0; i < n; i++)
    fork();                  /* ebeveyn DE çocuk DA devam eder */
printf("PID %6d  parent %6d\n", getpid(), getppid());
while (wait(NULL) > 0)       /* tüm çocukları topla: zombi yok */
    ;
```

Var olan her süreç sonraki her `fork()`'u yürütür. İlk fork'tan önce 1 süreç vardır; ondan sonra 2; ikinciden sonra 4; n'incisinden sonra 2^n. Bunların $2^n - 1$'ini `fork()` yaratmıştır (asıl süreç dışında hepsi) ve `printf` 2^n kez çalışır. $n = 3$ ile gerçek bir çalıştırma:

```
$ ./fork_count 3
PID   1939  parent   1937
PID   1938  parent   1937
PID   1940  parent   1937
PID   1941  parent   1938
PID   1942  parent   1939
PID   1943  parent   1938
```

```
PID   1937  parent   1903
PID   1944  parent   1941
```

Sekiz farklı süreçten sekiz satır. Ebeveyn sütunu ağacı ortaya çıkarır: asıl süreç 1937, 1., 2. ve 3. fork'larda 1938, 1939 ve 1940'ı yarattı; 1938 (1. fork'ta doğdu) kendisi de 2. ve 3. fork'ları yürüterek 1941 ve 1943'ü yarattı; ve böyle devam eder. Satırların sırası çalıştırmadan çalıştırmaya değişir, çünkü hangi sürecin önce çalışacağına zamanlayıcı karar verir.



Şekil 11: `./fork_count` 3'ün gerçek PID'lerle süreç ağacı.

Formül, çıktı satırları sayılarak doğrulanabilir:

```
$ for n in 0 1 2 3 4 5 6 7 8 9 10; do ./fork_count $n | wc -l; done | tr '\n' ' '
1 2 4 8 16 32 64 128 256 512 1024
```

Programın $n > 10$'u reddetmesinin iyi bir nedeni var: sınırsız fork eden bir döngü (bir "fork bombası") süreç tablosunu saniyeler içinde doldurur. Linux kullanıcı başına süreç sayısını sınırlar (`ulimit -u`); bu, Kısım 1'deki kaynak tahsisi hizmetinin bir örneğidir.

8 Özet

Yedi cümlede 3. hafta

1. İşletim sistemi kullanıcı için (kullanıcı arayüzü, program yürütme, G/Ç, dosya sistemleri, iletişim, hata algılama) ve kendisi için (kaynak tahsisi, hesap tutma, koruma ve güvenlik) hizmetler sunar.
2. Programlar bu hizmetlere sistem çağrılarıyla, normalde bir API ve ince bir kütüphane sarmalayıcısı üzerinden ulaşır; kabuk ve diğer sistem programları sıradan kullanıcı modu programlarıdır.
3. Her sistem çağrısının bir numarası vardır; tuzak komutu çekirdeğe tek bir sabit noktadan girer ve çekirdek numarayı sistem çağrısı tablosunda indeks olarak kullanır.
4. Parametreler yazmaçlarda, adresi bir yazmaçta duran bir bellek bloğunda ya da yığıtta aktarılır; x86-64 Linux numara için `rax`'i, parametreler için en çok altı yazmacı kullanır.
5. Süreç, text, data, bss, heap ve stack bölgeleri olan, yürütülmekte olan bir programdır ve süreç denetim bloğuyla tanımlanır.
6. Bir süreç yeni, hazır, çalışıyor, bekliyor ve sonlandı durumları arasında hareket eder; geçişlere sistem çağrıları ve kesmeler yol açar.

7. Bağlam değişimi bir PCB'yi kaydedip başka birini yükler ve saf ek yüktür, $s/(q+s)$; süreçler `fork()`, `exec()` ve `wait()` ile oluşturulur ve bir döngüdeki n fork 2^n süreç verir.

9 Sözlük

API (application programming interface) Uygulama programlama arayüzü: programların çağırabileceği belgelenmiş fonksiyonlar kümesi, ör. POSIX ya da Windows API.

ASLR (address space layout randomisation) Adres alanı düzeninin rastgeleleştirilmesi: bir sürecin bölgelerini her başlangıçta rastgele adreslere yerleştirmek.

Bağlam değişimi (context switch) Çalışan sürecin durumunu PCB'sine kaydedip başka bir sürecin durumunu onun PCB'sinden yüklemek.

BSS İlk değeri olmayan global değişkenleri tutan bölge; yükleme sırasında sıfırlarla doldurulur.

Daemon Açılışta başlayan ve kullanıcı arayüzü olmadan çalışan bir arka plan sistem programı (Windows'ta "service").

Hazır kuyruğu (ready queue) Çalışmaya hazır olup bir işlemci bekleyen süreçlerin listesi.

Kabuk (shell) Komut yorumlayıcısı; komutları okuyup çalıştıran bir sistem programı.

PCB (process control block) Süreç denetim bloğu: tek bir süreci temsil eden çekirdek veri yapısı.

PID (process identifier) Süreç kimliği: bir sürecin benzersiz numarası.

Sarmalayıcı (wrapper) Tek bir sistem çağrısı için yazmaçları dolduran ve tuzağı yürüten küçük kütüphane fonksiyonu.

Sistem çağrısı (system call) Bir programın çekirdeğe yaptığı, bir tuzak komutuyla girilen denetimli istek.

Sistem çağrısı tablosu (system-call table) Çekirdeğin sistem çağrısı numarasıyla indekslenen işleyici adresleri dizisi.

Sistem programı (system program) İşletim sistemiyle gelen kullanıcı modu programı (dosya yöneticisi, derleyici, `ls`, `ps`).

Süreç (process) Yürütülmekte olan bir program.

Yetim (orphan) Ebeveyni sonlanmış süreç; Linux'ta PID 1 ya da bir subreaper tarafından evlat edinilir.

Zombi (zombie) Çıkış durumu ebeveyni tarafından henüz toplanmamış, sonlanmış süreç.

10 Kendiniz Deneyin

Kendi makinenizdeki süreçlere bakın

Linux (laboratuvar bilgisayarları, WSL ya da bir sanal makine):

```
$ ps -ef | head           # tüm süreçler, PPID'leriyle
$ ps -o pid,ppid,stat,cmd  # süreçleriniz ve durumları
$ pstree -p | less         # süreç ağacı
$ cat /proc/$$/status      # kendi kabuğunuzun PCB'si
```

```
$ strace -c ls              # ls'in sistem çağrıları, türe göre
$ make && ./fork_count 4    # 3. hafta kod klasöründe
```

Windows: Komut İstemi'nde `tasklist` çalıştırın; Görev Yöneticisi'nde *Ayrıntılar* sekmesini açıp PID sütununu gösterin.

1. Kabuğunuzun PID'si nedir ve ebeveyni hangi süreçtir?
2. Şu anda R durumunda kaç süreç var? Neden bu kadar az?
3. Kabuğunuz şimdiye kadar kaç gönüllü bağlam değişimi yaptı? Birkaç komut çalıştırıp yeniden bakın.

11 Tekrar Soruları

Kavramsal

1. `printf()` ve `write()` örnekleriyle bir API fonksiyonu, bir sarmalayıcı ve bir sistem çağrısı arasındaki farkı açıklayın. `strlen()` bir sistem çağrısı yapar mı?
2. İşletim sistemine parametre aktarmanın üç yöntemini anlatın. x86-64 üzerindeki Linux hangisini kullanır ve neden ikisine birden ihtiyaç duyar?
3. Her sistem çağrısını altı kategoriden birine yerleştirin: `fork()`, `chmod()`, `ioctl()`, `getpid()`, `socket()`, `close()`.
4. Şunların her biri hangi bellek bölgesinde durur: global bir `int count;`, global bir `int max = 100;`, `main()`'in yerel bir değişkeni, `malloc()`'un döndürdüğü bellek, `main()`'in kodu?
5. Beş durumlu diyagramı çizin. Her geçiş için ona yol açan bir olay söyleyin. Bekliyor durumundan çalışıyor durumuna neden ok yoktur?
6. `fork()` ile `exec()` arasındaki fark nedir? `fork()` ne döndürür ve kime?
7. Zombi süreç nedir? Yetim nedir? Linux yetimlerin zombi olarak kalmamasını nasıl sağlar?

Sayısal

8. Bir bağlam değişimi $10\ \mu$s sürüyor. Zaman dilimi (a) 1 ms, (b) $50\ \mu$s ise işlemci zamanının yüzde kaçı ek yüktür?
9. Bir sistem saniyede 2000 bağlam değişimi yapıyor; her biri $4\ \mu$s sürüyor. İşlemcinin ne kadarı kaybedilir? Bu, saatte kaç işlemci-saniyesi eder?
10. Şu kod çalıştıktan sonra kaç süreç vardır ve `"hi"` kaç kez yazdırılır?
    `fork(); fork(); fork(); fork(); printf("hi\n");`
11. A süreci 2 ms hesap yapar, 6 ms diski bekler, sonra 2 ms hesap yapar. B süreci 4 ms hesap yapar. İkisi de $t = 0$'da hazırdır, önce A çalışır, tek bir işlemci vardır ve zaman dilimi sınırı yoktur. Toplam süreyi ve işlemci kullanımını hesaplayın: (a) B ancak A bittikten sonra başlarsa, (b) B, A beklerken çalışırsa.
12. x86-64 Linux'ta bir program `write(2, "Error\n", 6)` çağırıyor. `syscall` komutundan hemen önce `rax`, `rdi` ve `rdx`'te hangi değerler vardır ve bütün baytlar yazılırsa sonrasında `rax`'te ne olur?

Sayısal soruların yanıtları

8. (a) $10/(1000+10) \approx 0.0099 = 0.99\%$, yaklaşık %1. (b) $10/(50+10) \approx 0.167 = 16.7\%$: işlemcinin altıda biri boşa gider.
9. $2000\ \text{s}^{-1} \times 4 \times 10^{-6}\ \text{s} = 8 \times 10^{-3} = 0.8\%$. Saatte: $0.008 \times 3600\ \text{s} = 28.8$ işlemci-saniyesi.
10. Dört fork: $2^4 = 16$ süreç (15'i yeni yaratılmış) ve `"hi"` 16 kez yazdırılır.
11. (a) A $2+6+2 = 10$ ms sürer, sonra B 4 ms: toplam 14 ms; işlemci $2+2+4 = 8$ ms meşguldür, dolayısıyla kullanım $8/14 \approx 57\%$'dir. (b) A 0–2 arasında çalışır, B 2–6 arasında çalışır, işlemci 6–8 arasında boştadır, A 8–10 arasında çalışır: toplam 10 ms, kullanım $8/10 = 80\%$.
12. Önce: `rax` $= 1$ (`write`), `rdi` $= 2$ (standart hata), `rsi` $=$ dizginin adresi, `rdx` $= 6$. Sonra: `rax` $= 6$, yani yazılan bayt sayısı.

12 İleriye Bakış: 4. Hafta

- İş parçacıkları (threads): aynı belleği paylaşan, tek süreç içindeki birden çok yürütme akışı.
- Kullanıcı iş parçacıkları ve çekirdek iş parçacıkları; çoklu iş parçacığı modelleri (çoka-bir, bire-bir, çoka-çok).
- İşlemci zamanlamasının temelleri: işlemci–G/Ç patlama döngüsü (CPU–I/O burst cycle), kısa vadeli zamanlayıcı ve dağıtıcı (dispatcher).
- Zamanlama ölçütleri: işlemci kullanımı, iş hacmi (throughput), tamamlanma süresi (turnaround time), bekleme süresi ve yanıt süresi.

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Bölüm 2 (Kısım 2.1–2.5: hizmetler, kullanıcı arayüzleri, sistem çağrıları, sistem programları) ve Bölüm 3 (Kısım 3.1–3.3: süreç kavramı, zamanlama kuyrukları ve bağlam değişimi, süreçler üzerindeki işlemler).
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Bölüm 4, "The Abstraction: The Process": `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf`
  - Bölüm 5, "Interlude: Process API": `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`
  - Bölüm 6, "Mechanism: Limited Direct Execution": `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-mechanisms.pdf`